Wisdrop Studio: Research, Audit, v2 Plan

2026-11-06

Contents

Wisdrop Studio - extension research (2026-11-06) **1**
1. Short answer . . . 1
2. Comparison table . . . 2
3. What each family teaches . . . 3
   3.1 DOM-automation queue tools (Auto Flow family) . . . 3
   3.2 Stock-metadata tools (AutoMeta family) . . . 3
   3.3 API sidebars (Sider, HARPA, MaxAI, Monica, Merlin) . . . 3
   3.4 Agent extensions (Nanobrowser) . . . 4
4. Chrome MV3 facts (official docs) . . . 4
5. Chrome Web Store policy . . . 4
6. BYOK key security . . . 4
7. Terms of the AI sites on automation . . . 5
8. Sources (opened 2026-11-06) . . . 5

Wisdrop Studio v1 audit (2026-11-06) **7**
Fixed in v1.1 . . . 7
Open (not fixed now; reason) . . . 9
Checklist results . . . 10

Wisdrop Studio v2 plan (2026-11-06, for owner approval - NOT built yet) **12**
1. v2 in 3 lines . . . 12
2. Architecture (text diagram) . . . 12
3. Modules . . . 12
4. UI wireframes (text) . . . 13
5. Feature set (chosen from the research) . . . 13
6. What NOT to do (and why) . . . 14
7. Risks . . . 14
8. Phased build (effort = agent work sessions of ~1-2 h) . . . 14
9. Tie-in with agents 1-4 and the engine . . . 15

Wisdrop Studio - extension research (2026-11-06)

Scope: what similar Chrome extensions already do, how they are built, what breaks for their users, what Chrome and the AI sites allow, and what that means for Wisdrop Studio. Every claim has a numbered source [n]; all pages were opened on 2026-11-06 (a few blocked pages are marked "via search excerpt").

1. Short answer

1. **Two families exist.** (a) *DOM-automation queue tools* that drive a website you are signed into (Auto Flow / TurboFlow / AutoFlow for Google Flow, AutoMeta and MetaForge for Adobe Stock, MidBot / AutoJourney for Midjourney). (b) *API sidebars / agents* that call model APIs with a key (Sider, HARPA, Monica, MaxAI, Merlin, BYOK AI Chat, Nanobrowser). Wisdrop Studio is family (b) plus a GitHub/Modal engine, which is the safer family.

2. **DOM automation is fragile and often against the site's terms.** Reviews of Auto Flow are mostly negative: crashes, random stops, breaks after Flow updates [11]. AIPRM and Superpower ChatGPT broke on many ChatGPT UI updates [20][31][32]. Midjourney's terms forbid automated tools and allow bans [33][34]; Google's terms forbid automated access against machine-readable rules [19]; one Flow tool's own terms warn Google may suspend your account [35].
3. **The winning UX ideas are simple:** side panel; paste or import a list -> queue rows with status; pause / resume / reorder / retry-failed; per-batch settings; import/export JSON; auto-named outputs; speed control that slows down on rate limits; a detailed activity log [10][26][27][12].
4. **Chrome MV3 rules that matter:** service worker dies after 30 s idle or 5 min per call, so long jobs must persist state to storage and resume [1]; `storage.local` is 10 MB and readable by content scripts unless the access level is changed [3]; alarms fire at most every 30 s [5]; optional host permissions requested at runtime from a user gesture [6]; no remote code, no eval, no interpreters of fetched commands [7]; narrowest permissions [9].
5. **BYOK security reality:** extension storage is not encrypted at rest; keep keys out of `storage.sync`, out of content scripts and logs, and encourage a per-app key with a spend cap [23][29]. Anthropic needs the `anthropic-dangerous-direct-browser-access: true` header for browser calls [13]; Ollama blocks extension origins unless `OLLAMA_ORIGINS` includes `chrome-extension://*` [22]; Google says never ship Gemini keys in client code (BYOK typed by the user is the accepted pattern) [16].

2. Comparison table

Tool	What it does	UI	How it works	Price	What users complain about
Auto Flow (duckmartians) [10]	batch images/videos on Google Flow, auto-save	panel + Flow tab	DOM automation in your signed-in Flow tab; uses your Flow credits; up to 1000 threads on top plan	free / $3-$10 per month	(see Auto Flow Veo below)
Auto Flow - Auto Veo (30K users) [11]	same	panel	DOM automation	$10/mo, free 10 prompts/day	crashes, random stops, auto-download fails, breaks after Flow updates, support
TurboFlow (10K users, 4.4) [26]	queue of batches, auto-chain, upscale, download	panel	DOM automation; retries, slows on rate limits, failed list, activity log, JSON import/export	paid tiers	(good rating; relies on Flow UI)
AutoFlow - Batch [27]	queue, pause/resume/reorder, retry	side panel	DOM automation	free/pro	small user base
AutoMeta for AdobeStock (1-2K users) [17]	titles + keywords on Adobe Stock upload page, "Get All Metadata" batch	in-page buttons	content script on contributor.stock.adobe.com + Gemini API (BYOK, free tier)	free	a hard-coded Gemini model was retired, so it broke (search excerpt [37])
MetaForge (Adobe Stock) [28]	same, 3 providers, key rotation	side panel	content script + BYOK (Gemini/OpenAI/Groq)	free	key rotation to dodge limits is a policy grey zone

Tool	What it does	UI	How it works	Price	What users complain about
MidBot / AutoJourney (Midjourney) [34][36]	CSV prompt queue, auto upscale + download	popup / panel	DOM automation of midjourney.com / Discord	paid	ToS forbids automation; user-reported bans; breaks when UI changes
Sider (5M users) [21]	multi-model chat, summaries, agent "Hand"	sidebar	vendor cloud credits	free 30 basic credits/day, paid tiers	credit limits unclear, "unlimited" not unlimited (search excerpt [38])
HARPA AI (400K users, 4.5-4.7) [18]	page-aware chat, monitors, automation	sidebar (Alt+A)	vendor cloud or BYOK or "web session"	$12 / $19 per month (annual), $240 life	steep learning curve, broad trust needed
MaxAI [39]	sidebar chat, rewrite	sidebar	vendor cloud	$12/mo yearly, $30 monthly	-
AIPRM / Superpower ChatGPT [20][31][32]	prompt library injected into ChatGPT	overlay inside chatgpt.com	content script DOM injection	freemium	UI broken after each ChatGPT update; conflicts between extensions
BYOK AI Chat (open source) [30]	multi-provider chat	side panel	API calls from service worker, `chrome.storage.local`	free	small, basic
Nanobrowser (13K stars) [14][15]	web agent, any model	side panel	agent loop + `chrome.debugger` (CDP) control layer; Ask mode before submit/send/delete/upload; stops after 3 failed checks; only while browser open	free, BYOK	needs tool-calling models

3. What each family teaches

3.1 DOM-automation queue tools (Auto Flow family)

- Pattern: side panel queue + content script that types the prompt, clicks generate, waits, downloads, renames [10][12][26].
- Good: rows with status, threads/concurrency limit, sequential mode (output of row n is the reference for row n+1), character / reference libraries, per-batch settings, import/export JSON, retry failed only, random delay [10][26][36].
- Bad: every site redesign breaks them [11][20]; "unlimited" plans still burn the user's own credits [10]; account risk [33][35]. AIPRM even tells users to uninstall other ChatGPT extensions because injected UIs fight each other [31].
- **Lesson for Wisdrop:** copy the queue UX, not the DOM driving. Our engine already runs on GitHub Actions + Modal via APIs, so we never need to puppet a site.

3.2 Stock-metadata tools (AutoMeta family)

- Pattern: content script on one site + BYOK vision model to fill forms [17][28].
- Lessons: never hard-code one model name (AutoMeta broke when a preview model was retired [37]); let the user pick from a fetched model list (v1 already has Fetch models). Key rotation across many free keys to beat rate limits is abuse-prone - do not do it.

3.3 API sidebars (Sider, HARPA, MaxAI, Monica, Merlin)

- Pattern: sidebar chat, prompt/command library, page-aware context, credits [18][21][39].

- Complaints are about money transparency, not tech: credit systems users do not understand [21][38].
- **Lesson:** show cost live in tokens and dollars, with a hard cap per run and per day.

3.4 Agent extensions (Nanobrowser)

- One agent per chat, real tools, "Ask" mode that pauses before any irreversible action, site Allow/Ask/Block list, stops after 3 failed checks, long chats summarised, chat history with re-run [14][15].
- Matches the agent guidance from Anthropic: clear stopping conditions (max iterations), human checkpoints, simple loops [24]; small, well-named, namespaced tools that return token-efficient, meaningful results and clear errors [41].

4. Chrome MV3 facts (official docs)

- **Service worker lifetime:** terminated after 30 s idle, if one event/API call takes over 5 min, or a fetch takes over 30 s to answer; global variables are lost - persist to `chrome.storage` or IndexedDB [1]. Long agent runs must not live only in SW memory.
- **Side panel:** extension page with full Chrome API access; `setPanelBehavior({openPanelOnActionClick:true})`; `sidePanel.open()` needs a user gesture; per-tab or global panels; onOpened/onClosed events (Chrome 141/142) [2]. A side panel page lives while open, so a loop running there survives SW sleep but dies when the panel closes.
- **Storage:** `local` 10 MB (more with `unlimitedStorage`), exposed to content scripts by default (change with `setAccessLevel`); `session` 10 MB in memory, cleared on restart, not exposed to content scripts by default; `sync` ~100 KB and syncs to the Google account - wrong place for secrets [3].
- **Offscreen documents:** hidden DOM for blobs, DOMParser, audio, clipboard etc.; only `runtime` API inside; one open at a time [4]. Useful later for big-file hashing or audio checks.
- **Alarms:** min period 30 s (Chrome 120+); missed alarms fire once after sleep; re-create on startup [5]. Use for polling GitHub runs instead of setTimeout in the SW.
- **Optional permissions:** request at runtime in a user gesture; `https://*/*` in optional hosts allows runtime-discovered hosts [6].
- **activeTab:** temporary access to the current tab after the user invokes the extension; lost on navigation [42].
- **CSP:** extension pages minimum `script-src 'self' 'wasm-unsafe-eval'; object-src 'self'`; cannot be relaxed (no unsafe-eval) [8]. Avoid innerHTML; build DOM nodes; never send secrets to content scripts [43].

5. Chrome Web Store policy

- **No remote code:** no remote `<script>`, no eval of fetched strings, no interpreter for fetched commands "even if fetched as data" [7]. *Note for us:* loading agent **prompts** (text instructions to an LLM) from GitHub is data for a model, not code; the extension's own tool set is fixed in the package. Keep it that way: never let the repo ship JS or a "command language" the extension executes.
- **Permissions:** narrowest permissions; no "future proofing" [9].
- **Limited use of user data:** use data only for the single purpose; transfers only to provide the feature; no humans reading data [44].

6. BYOK key security

- Extension storage is not encrypted at rest; anyone with the profile can read it; keep keys out of `storage.sync`, out of content scripts; encourage a capped per-app key [23].
- Better builds keep keys only in privileged contexts, encrypt with WebCrypto AES-GCM (non-extractable key) and wipe on sign-out, while stating honestly that encryption only raises the bar [29].
- Anthropic: browser calls need `anthropic-dangerous-direct-browser-access: true`; risk is a leaked embedded key, acceptable when the user brings their own [13]. Anthropic's monthly spend cap returns 429 with no retry-after; retrying does not help [40].
- Gemini: never hard-code keys in client code; use a backend for shipped apps [16]. (BYOK typed by the owner is the accepted exception.)
- Ollama: allows only localhost/127.0.0.1 origins; extensions need `OLLAMA_ORIGINS=chrome-extension://*` (or the exact extension id) [22].
- GitHub REST: 5,000 req/h per token; secondary limits answer 403/429 - honour `retry-after` and `x-ratelimit-remaining: 0` [25].

7. Terms of the AI sites on automation

- **Midjourney:** "You may not use automated tools to access, interact with, or generate Assets through the Services"; may suspend/ban at any time [33][34].
- **Google (Flow, Gemini app):** no "automated means to access content ... in violation of the machine-readable instructions" plus the Generative AI Prohibited Use Policy [19]. Flow automation tools disclaim that Google may rate-limit, suspend or close accounts [35].
- **ChatGPT / Claude web apps:** extensions that inject UI break on each redesign [20][32]; automating the consumer chat UI is not needed when APIs exist.
- **Conclusion:** use official APIs (model APIs, GitHub, Modal). A DOM-assist mode, if ever built, must be *assist only* (fill a box the owner focused; owner presses send), opt-in per site, and never loop unattended.

8. Sources (opened 2026-11-06)

1. Chrome: Extension service worker lifecycle - https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle
2. Chrome: chrome.sidePanel API - https://developer.chrome.com/docs/extensions/reference/api/sidePanel
3. Chrome: chrome.storage API - https://developer.chrome.com/docs/extensions/reference/api/storage
4. Chrome: chrome.offscreen API - https://developer.chrome.com/docs/extensions/reference/api/offscreen
5. Chrome: chrome.alarms API - https://developer.chrome.com/docs/extensions/reference/api/alarms
6. Chrome: chrome.permissions API - https://developer.chrome.com/docs/extensions/reference/api/permissions
7. CWS: MV3 additional requirements (remote code) - https://developer.chrome.com/docs/webstore/program-policies/mv3-requirements
8. Chrome: Manifest content_security_policy - https://developer.chrome.com/docs/extensions/reference/manifest/content-security-policy
9. CWS: Use of permissions - https://developer.chrome.com/docs/webstore/program-policies/permissions
10. Auto Flow README (duckmartians) - https://github.com/duckmartians/Auto-Flow
11. ExtWise: Auto Flow - Auto Veo review analysis - https://extwise.com/extension/auto-flow-auto-veo-nano-banana-pro/
12. AutoFlow blog: batch generate on Google Flow - https://www.auto-flow.studio/blog/how-to-batch-generate-ai-videos-google-flow (search excerpt)
13. Simon Willison: Anthropic CORS header - https://simonwillison.net/2024/Aug/23/anthropic-dangerous-direct-browser-access/
14. Nanobrowser v0.3.0 release - https://github.com/nanobrowser/nanobrowser/releases/tag/v0.3.0
15. Nanobrowser docs - https://nanobrowser.ai/docs
16. Google AI: Gemini API keys - https://ai.google.dev/gemini-api/docs/api-key
17. CWS: AutoMeta for AdobeStock - https://chromewebstore.google.com/detail/autometa-for-adobestock/gofdlplbcbhanomlnnmih
18. HARPA AI pricing - https://harpa.ai/pricing
19. Google Terms of Service - https://policies.google.com/terms?hl=en-US
20. AIPRM forum: UI broken by ChatGPT update 2025-04-03 - https://forum.aiprm.com/t/solved-aiprm-ui-partially-broken-for-some-users-due-to-chatgpt-update-on-april-3-2025/78675
21. Sider credits FAQ - https://sider.ai/help-center/faqs/sider-credit-issue
22. Ollama FAQ (origins) - https://docs.ollama.com/faq
23. Omni Sidekick: how your API key is stored - https://omnisidekick.app/trust/api-key-security
24. Anthropic: Building effective agents - https://www.anthropic.com/engineering/building-effective-agents
25. GitHub: REST API rate limits - https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api
26. CWS: TurboFlow - https://chromewebstore.google.com/detail/turboflow-auto-flow-on-go/bnfjjfgjpdagdiiiifdkjbjnadgdjaoe
27. CWS: AutoFlow - Batch for Google Flow - https://chromewebstore.google.com/detail/autoflow-%E2%80%94-batch-for-goog/fohjehblomodhghkjbemblplknjinaln
28. ExtScope: MetaForge by TAS - https://extscope.org/extension/mgkncjjdibphdjhfcjfakihecmkiecmc
29. Inquiso: security and threat model - https://inquisoai.com/docs/design/05-security/
30. BYOK AI Chat (GitHub) - https://github.com/alvescrafter/byok-ai-chat
31. AIPRM: Chrome extensions that break AIPRM - https://www.aiprm.com/en-ie/blog/when-chrome-extensions-break-aiprm/
32. Superpower ChatGPT release notes - https://spchatgpt.com/release-notes/
33. Licenseware: Midjourney ToS rule (quotes ToS s.1) - https://wiki.licenseware.io/catalog/rules/midjourney-no-automated-access-or-generation/
34. Extino: Midjourney bulk prompts + ToS - https://extino.net/blog/midjourney-bulk-prompts-batch-generate
35. Flow Connector terms - https://flow-connector.flowconnector.workers.dev/terms

36. AutoJourney docs (ban disclaimer, search excerpt) - https://docs.autojourney.ai/en-US/guide
37. Chrome-Stats: AutoMeta reviews (search excerpt; page blocked) - https://chrome-stats.com/d/gofdlplbcbhanomlnnmihfakocood
38. Trustpilot: Sider reviews (search excerpt; page blocked) - https://www.trustpilot.com/review/sider.ai
39. MaxAI pricing - https://www.maxai.co/pricing/
40. Anthropic API rate limits - https://docs.anthropic.com/en/api/rate-limits
41. Anthropic: Writing tools for agents - https://www.anthropic.com/engineering/writing-tools-for-agents
42. Chrome: activeTab permission - https://developer.chrome.com/docs/extensions/develop/concepts/activeTab
43. Chrome: Stay secure - https://developer.chrome.com/docs/extensions/develop/security-privacy/stay-secure
44. CWS: Limited Use - https://developer.chrome.com/docs/webstore/program-policies/limited-use

Wisdrop Studio v1 audit (2026-11-06)

Audited: every file in `extension/` (manifest.json, background.js, sidepanel.html/css/js, lib/agent.js, lib/github.js, lib/providers.js, lib/store.js, test/run.js, README). Checked against `docs/EXTENSION_RESEARCH.md` (sources [n] there). Checks run: `node --check` on all 7 JS files (pass); `node extension/test/run.js` (11 groups pass, incl. new v1.1 group). Headless Chromium: **not available in the sandbox** (no chromium/playwright) - real-Chrome load test still open (owner).

Result: v1 -> **v1.1** (manifest 1.1.0). 11 issues fixed, 14 open items (13 to do in v2 or accepted, 1 info).

Severity: **H** = can cause damage or spend / lose work; **M** = wrong behaviour or weak guard; **L** = polish.

Fixed in v1.1

#	Sev	Area	Finding	Fix
F1	H	write guard	`checkWritePath` trusted the Episode field. If the field said `docs` (or was empty) the agent could write anywhere in the repo, e.g. `AGENTS.md` or workflows.	`checkEp()`: episode must match `episodes/<name>`; enforced in writes, dispatch, and every panel action. Test added.

#	Sev	Area	Finding	Fix
F2	H	agent protocol	`extractAction` cut the JSON at the first “`fence. Any write_file whose markdown content has a code block was truncated or failed (most of our docs have code blocks). | Scan braces from the opening fence over the rest of the text (string-aware). Test with fenced content added. | | F3 | H | GPU spend | dispatch_episode could send max_gpu_seconds = “NaN” (non-numeric model output) or 0/negative; workflow behaviour then undefined. Empty test_lines from the agent = whole-episode build. | dispatchInputs(): integer within owner cap (default 120 s), else the cap; line lists must match 0-15,3,7; agent cannot dispatch a whole-episode build (owner uses Run tab, which asks). Tests added. | | F4 | H | lost work | Agent session lived only in panel memory; closing the panel or a crash lost the whole chat, step log and a pending gate. (The loop runs in the side panel page, not the service worker, so SW sleep [1] is not the killer - panel close is.) | Session (messages, tokens, log tail, pending gate) saved to chrome.storage.local after every step, key-shaped strings redacted, capped ~4 MB (10 MB quota [3]); restored on open with a note; New chat clears it. | | F5 | M | rate limits | No retry on 429/5xx /529 from model APIs; one overload error killed a run. | fetchRetry: 4 tries, exponential backoff + jitter, honours retry-after (max 30 s);	

#	Sev	Area	Finding	Fix
F11	L	weak passphrase	No minimum length.	>= 10 characters.

Open (not fixed now; reason)

#	Sev	Area	Finding	Plan
O1	M	keys at rest	Default vault is plain `chrome.storage.local` (passphrase optional); storage is not encrypted at rest [23].	v2: nudge passphrase on first save; recommend a capped, per-app key per provider.
O2	M	long runs	Loop dies when the panel closes; v1.1 restores the chat but does not resume a run automatically. GitHub run polling is manual.	v2 job queue: jobs persisted in storage, SW `alarms` (>= 30 s [5]) poll GitHub runs and notify; panel resumes the job on open.
O3	M	cost cap	Budget is tokens only; no $ estimate, no daily cap across chats.	v2: price table per model (owner-editable), $ per run and per day, hard stop.
O4	M	context growth	Every tool result (up to 12k chars) stays in the chat; long runs overflow small models.	v2: summarise old tool results (keep owner messages verbatim, as Nanobrowser does [14]).
O5	M	prompt injection	`fetch_url` and repo text can instruct the model. Guards: writes only inside the episode, dispatch needs owner click, gates are PR merges.	v2: mark tool output as data in the protocol; per-site Allow/Ask/Block list; show the domain in every fetch card.
O6	M	permissions	`optional_host_permissions` includes `https://*/*` (needed for custom base URLs and fetch_url) - CWS reviewers may ask why [9]. `scripting`+`activeTab` only serve "Insert prompt".	If we publish: justify in listing, or drop Insert-prompt and `scripting`. Personal use (unpacked): fine.
O7	L	dispatch run id	After dispatch, "latest run" may be someone else's run.	v2: match run by `created_at` > dispatch time and `display_title`/inputs.
O8	L	abort	Stop aborts the model call but not an in-flight GitHub call.	Pass the signal into GitHub client.

#	Sev	Area	Finding	Plan
O9	L	blobs	Preview video loaded as a blob through the contents API; files > 100 MB fail.	Link to the run artifact instead of blob for big files.
O10	L	a11y	Tabs, labels and live regions OK; confirm cards are `alertdialog` without focus trap; no visible focus style check; contrast not measured.	v2 UI pass with axe in a real Chrome.
O12	L	model names	Default model ids may be retired (AutoMeta broke this way [37]).	v1 already has Fetch models; v2 warns when a saved model disappears from the list.
O13	L	real Chrome	Never loaded in a real Chrome; no headless Chromium in sandbox.	Owner: load unpacked, Keys -> Test; report errors.
O15	M	workflow (engine side)	`episode.yml` puts `${{ inputs.* }}` straight into `run:` shell lines (e.g. `--force "${{ inputs.redo_images }}"`); a crafted input could inject shell. v1.1 now validates every agent dispatch input (digits/ranges, picture ids), which closes it from the extension side.	Engine fix later: pass inputs via `env:` and quote. Not changed in this run (workflow edit = engine change, needs a test run).
O14	info	secrets in commits	Agent writes are scanned for key shapes and refused; redaction in log/UI. OK. Modal keys never reach the extension (GitHub secrets). OK.	keep

Checklist results

- Manifest permissions: `sidePanel, storage, scripting, activeTab, clipboardWrite`; host `api.github.com` only; providers optional at runtime. Least privilege acceptable; see O6.
- CSP: `script-src 'self'; object-src 'self'` - at the MV3 minimum [8]. OK.
- Remote code: none; prompts from GitHub are LLM text, tools are fixed in the package [7]. OK.
- Key storage: local only, never sync; AES-GCM + PBKDF2 250k when a passphrase is set; redaction. See O1.
- SW sleep: background.js only sets panel behaviour; the loop runs in the panel page, so SW sleep does not kill it [1]. Panel close does (F4 restore, O2 resume).
- CORS: Anthropic header present [13]; Gemini key sent in `x-goog-api-key` header (not URL); Ollama README tells owner to set `OLLAMA_ORIGINS=chrome-extension://*` [22] (better: the exact extension id).
- GitHub token: README asks for fine-grained, this repo only, Contents/Actions/PRs R/W + Metadata R. OK.
- Error handling: per-tool errors return to the model as `ERROR:` text; two protocol failures stop the loop. OK.
- Gate enforcement: model `gate` = hard stop; owner approval in Review = PR merge (gates.yml). `state.json` never

written by agent. OK.

- GPU spend: every agent dispatch asks the owner (inline card) and is capped (F3). OK.
- XSS: all model/repo text goes through `textContent`; links only for `http(s)`; no innerHTML. OK [43].

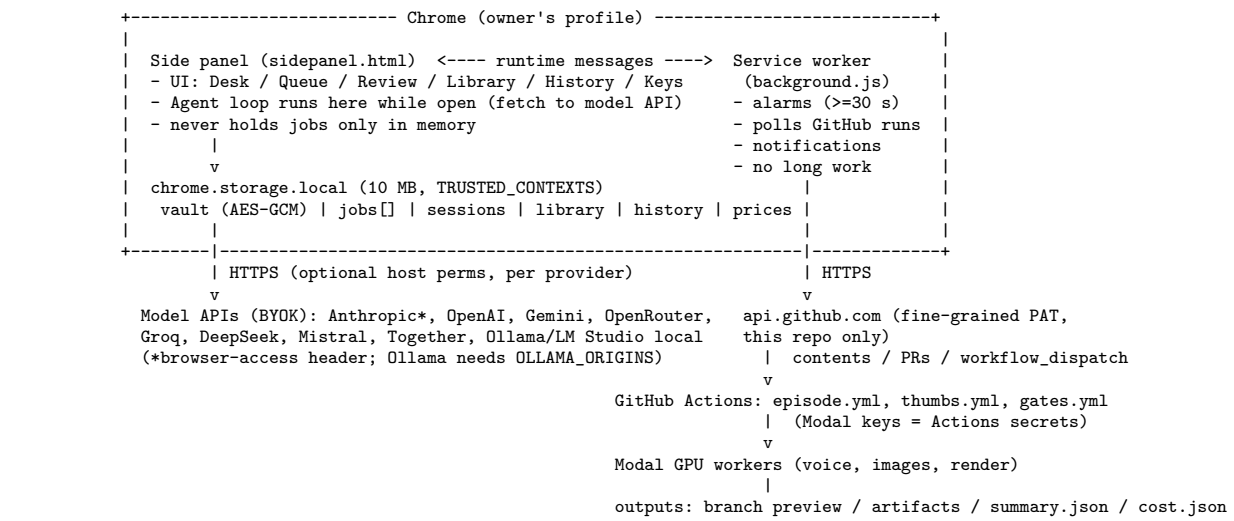
Wisdrop Studio v2 plan (2026-11-06, for owner approval - NOT built yet)

Based on `docs/EXTENSION_RESEARCH.md` (sources [n]) and `docs/EXTENSION_AUDIT.md`. Goal: turn the v1.1 chat panel into a **production desk** for one episode: queue of jobs that survives closing Chrome, per-chapter batch runs, a review grid, hard cost caps, run history - all through official APIs (model APIs, GitHub, Modal via GitHub Actions). No puppeting of AI websites.

1. v2 in 3 lines

1. **Job queue with resume:** every agent step and every GitHub run is a job saved in storage; alarms poll runs; the panel picks up where it stopped.
2. **Chapter batch desk + review grid:** run voice/render per chapter or per shot list, see every clip/line/fact in one grid, approve or send back per item.
3. **Money and safety rails:** $ caps per run/day, Ask-before-act for writes/dispatch/merge, site Allow/Ask/Block, prompt library and run history with export.

2. Architecture (text diagram)



Rules: the service worker only schedules and polls (it dies after 30 s idle [1]); state always in storage; the agent loop runs in the panel page and checkpoints after each step; nothing executes code fetched from the repo [7].

3. Modules

Module	File	Does
store	lib/store.js	vault (keep), `jobs`, `sessions`, `library`, `history`, `prices` stores; quota check (`getBytesInUse`) with pruning
providers	lib/providers.js	keep adapters + retry; add streaming (optional), price lookup, model-gone warning
github	lib/github.js	keep; add signal/abort, run matching by dispatch time + title, artifact listing for big files
agent	lib/agent.js	keep JSON protocol; add tool-output-as-data wrapper, history summariser, Ask policy per tool, step checkpoints

Module	File	Does
queue (new)	lib/queue.js	job model `{id, kind: agent_step|dispatch|poll|review, ep, chapter, inputs, status: queued|running|waiting_owner|done|failed|cancelled, tries, cost, created, updated}`; pause/resume/reorder/retry-failed; one heavy run at a time (matches `concurrency: heavy-worker`)
cost (new)	lib/cost.js	token -> $ from owner-editable price table; caps per run / per day / per episode; reads `cost.json` for GPU $; hard stop + banner
library (new)	lib/library.js	prompt snippets and agent "recipes" (e.g. "Chapter 3 voice test 0-15"); import/export JSON
background	background.js	alarms: poll running GitHub jobs every 1 min, update jobs, notify on done/failed/gate; re-create alarms on startup [5]
ui	sidepanel.* split into panels/desk.js, queue.js, review.js, history.js, keys.js	plain JS, no build step, no remote code
tests	test/*.js	unit (mock fetch) + a manual real-Chrome checklist

4. UI wireframes (text)

```
DESK (default)
+-----------------------------------+
| ep01-onecoin  state: chapters     |
| Today $0.42 / cap $2.00  [===  ]  |
|-----------------------------------|
| Chapter   lines  voice  render QA |
| ch1 Hook  0-15   ok     ok     ok |
| ch2 ...   16-38  ok     run    -  |
| ch3 ...   39-61  [Voice][Render]  |
|-----------------------------------|
| Agent: [02 Director v] [Ask mode] |
| > chat ...                        |
| [Send] [Stop]  tokens 12k  $0.05  |
+-----------------------------------+

HISTORY
+-----------------------------------+
| date  agent/run  result  $  link  |
| [Export JSON] [Export markdown]   |
+-----------------------------------+
```

```
QUEUE
+-----------------------------------+
| [Pause all] [Retry failed] [Export]|
| # job            status   cost    |
| 1 agent 02 ch3   done     $0.03   |
| 2 voice ch3 0-15 running  GPU 40s |
| 3 render ch3     queued   ~60s    |
| 4 gate chapters  WAITING OWNER    |
|  [up][down][cancel] per row       |
+-----------------------------------+

REVIEW GRID
+-----------------------------------+
| filter: ch3 | clips | lines | facts|
| [clip 39][clip 40][clip 41] ...   |
|  each: play, FM flags, Approve,   |
|  Send back + note                 |
| [Approve chapter = merge gate PR] |
+-----------------------------------+
```

5. Feature set (chosen from the research)

Feature	From	Why for us
Job queue with pause / resume / reorder / retry-failed, persisted	TurboFlow, AutoFlow [26][27]; MV3 lifecycle [1]	closing Chrome must not lose a chapter run
Per-chapter batch runs (voice/render per chapter, redo lists)	Auto Flow batches + sequential mode [10]	matches `chapters.json` and `episode.yml` test_lines
Review grid (clips, voice lines, fact rows) with per-item approve / send back	queue status views [26]; our gates	faster owner review; gate = PR merge stays
Cost caps ($ per run/day/episode) + live meter	Sider/HARPA credit complaints [21][38]	no surprise bills; GPU cap already in workflow
Ask mode before write / dispatch / merge; site Allow/Ask/Block for fetch	Nanobrowser [14][15]	human-in-the-loop by default

Feature	From	Why for us
Stop rules: max steps, 3 failed checks, token + $ budget	Nanobrowser [14]; Anthropic [24]	no runaway loops
Prompt library + recipes, import/export JSON	AIPRM-style library [20], TurboFlow JSON [26]	reuse of good agent instructions
Run history + export (JSON/markdown)	Nanobrowser chat history [14]	audit trail, handover
Activity log with filters	TurboFlow [26]	debugging
Model list fetch + "model retired" warning	AutoMeta breakage [37]	no silent failure
Notifications when a run finishes or a gate waits	-	owner can leave the desk
Optional DOM-assist (Insert prompt into focused box only)	v1	keep, opt-in, assist only

6. What NOT to do (and why)

- **No unattended DOM automation of Flow, Midjourney, ChatGPT, Claude, Kling, Runway web apps.** Breaks on every redesign [11][20][32]; violates terms and risks bans [19][33][35]. Our engine has APIs.
- **No API-key rotation across many free keys** to dodge limits (abuse; MetaForge pattern [28]).
- **No vendor server / cloud credits / accounts.** BYOK only; nothing to breach; CWS limited-use stays simple [44].
- **No remote code or repo-defined command language** executed by the extension [7]; prompts stay text for the model.
- **No `storage.sync` for anything secret** [3][23]; no keys in logs, URLs or commits.
- **No auto-merge of gate PRs and no auto-upload to YouTube.** Owner approvals stay manual (NEVER BEND).
- **No 1000-thread parallelism.** One heavy run at a time (matches workflow concurrency) - cost and quota safety.
- **No framework/build step** unless the UI outgrows plain JS; keeps the package reviewable.

7. Risks

Risk	Mitigation
Panel closed mid-agent-run	checkpoint each step (done in v1.1); resume button; GitHub runs continue server-side and alarms track them
Storage quota (10 MB) with long sessions	prune history, cap sessions ~4 MB, `getBytesInUse` meter, export then clear
Model API price changes	owner-editable price table; caps in $ and tokens
Prompt injection from fetched pages	tool output wrapped as data; writes only in episode folder; dispatch/merge always Ask
GitHub secondary rate limits	GET retries with retry-after; poll every 60 s max
Workflow input injection (audit O15)	extension validates inputs (done); engine: move inputs to `env:`
Real-Chrome bugs untested in sandbox	owner smoke test of v1.1 before v2 build

8. Phased build (effort = agent work sessions of ~1-2 h)

Phase	Content	Effort	Done when
P0	Owner loads v1.1 in Chrome, smoke test (Keys test, one agent read, one 0-15 s test dispatch)	owner 15 min	no blocking errors

Phase	Content	Effort	Done when
P1	queue.js + background alarms + job persistence + resume; run matching	2 sessions	close panel mid-run, reopen, job continues and finishes
P2	Desk: chapter table from chapters.json, per-chapter Voice/Render, redo lists	1 session	ch1 voice test via desk
P3	cost.js: price table, $ meter, caps; GPU $ from cost.json	1 session	cap stops a run
P4	Review grid + per-item send back (PR comment) + approve chapter (merge)	2 sessions	review ch1 entirely in grid
P5	Ask policy per tool, site Allow/Ask/Block, tool-output-as-data, history summariser	1 session	injection test page cannot trigger a write
P6	Library + History + import/export; notifications	1 session	export history JSON
P7	a11y pass, real-Chrome checklist, README v2, zip	1 session	owner sign-off

Total ~9 sessions + owner tests. Each phase: tests, commit, zip, short owner note.

9. Tie-in with agents 1-4 and the engine

- **Agent 1 Research+Writer:** Desk runs it to `topic` and `script` gates; facts table feeds Review grid (sources column, R01 flags from fm_check).
- **Agent 2 Director:** produces `chapters.json` -> Desk chapter table; `scene_sheet` and `chapters` gates shown as queue rows waiting for owner.
- **Agent 3 Builder:** each chapter = jobs: voice test (episode.yml test_lines) -> render preview -> fm_check summary -> Review grid.
- **Agent 4 Voice+Finish:** final build job (whole episode) only from the owner's Desk button with $ cap; `final` gate = PR merge; upload stays manual.
- **Engine:** unchanged contract: `workflow_dispatch` inputs (ep, test_lines, preview_seconds, redo_images, redo_lines, max_gpu_seconds) -> `summary.json` / `cost.json` on branch `preview`; gates via `gates.yml` PR merges; Modal keys stay GitHub secrets.